

Laboratory guide nº 1 – Discrete event simulation

Objectives

The aim of this laboratory work is to study the techniques for generating random variables and stochastic processes, and for programming discrete event simulators and analyze its output data.

Exercises

Before start performing the exercises see the video “Revisions of probabilities” (revprob)

A. Random number generators (class 1)

Before performing these exercises see the video “Random number generators” (rng)

1. Plot a histogram of 1000 random numbers (numbers uniformly distributed in $[0,1]$) using the `hist()` and `runif()` functions of R. Is the result according to your expectations? **Hint:** If you want to learn what a function does, use the Help tab of RStudio (bottom right corner), or type `help(<function name>)` at the Console.
2. Using the `runif()` function of R, verify that the sequence of numbers generated by a pseudo-random number generator is in fact deterministic. Explain the procedure that you have developed to make this verification. **Hint:** Use the `set.seed()` function.

B. Generation of random variables and stochastic processes (class 1)

Before performing these exercises see the video “Generation of random variables and stochastic processes” (genvar)

3. Develop an R script to generate Bernoulli distributed numbers using a random number generator, i.e., the `runif()` function. Plot a histogram of 1000 Bernoulli numbers to check (informally) that the generator is operating correctly. Try different values of the success probability. **Hint:** Use the `ifelse()` function.
4. Develop an R script to generate exponentially distributed numbers using a random number generator, i.e., the `runif()` function. Plot a histogram of 10000 exponentially distributed numbers to check (informally) that the generator is operating correctly. Superpose the histogram and the pdf of the exponential distribution. **Hints:** Use the `dexp()` and `seq()` functions to obtain the pdf values of the exponential distribution, and the `lines()` function to obtain its graph; if you run `hist()` and afterwards `lines()`, the two graphs will get

superposed. The comparison is only possible if you use the `prob=TRUE` option in the `hist()` function; Why?

5. Develop an R script to generate the arrival times of a Poisson process. Perform a simple test to verify that the sample rate is according to the population rate. **Hints:** *A Poisson process has exponentially distributed inter-arrival times; you can use the `rexp()` function to generate exponentially distributed numbers. For a more compact solution use the `cumsum()` function.*

C. Programming discrete event simulators (class 2)

Before performing these exercises see the video “Programming discrete event simulators” (desprog)

6. Consider the simulator for estimating the average delay in queue of an M/M/1 system whose R code is presented in Annex A. Modify this simulator to estimate the average delay in system and the server utilization. Compare the values obtained with the theoretical ones (see Annex B). Try different values (at least two) of λ/μ such that $\lambda/\mu < 1$ and the performance is significantly different. **Hint:** *When debugging code, it may be useful to print the values of some data structures; use the `print()` or `sprintf()` functions for this purpose.*
7. Develop a simulator for estimating the average delay in queue of an M/M/2 system. Compare the values obtained with the theoretical ones (see Annex C). Try different values (at least two) of λ/μ such that $\lambda/2\mu < 1$ and the performance is significantly different. **Hint:** *Consider as a starting point the M/M/1 simulator of Annex A.*

D. Estimation of probabilities (class 3)

Before performing this exercise see the video “Estimation of probabilities” (probest)

8. Develop an R script to simulate the tossing of five biased coins and estimate the probability of getting two heads when five coins are tossed; consider that the probability of getting a head is 0.7. Compare the estimate obtained through simulation with the exact value. **Hint:** *To estimate the probability, you will have to repeat the experiment many times, say 1000.*

E. Revisions of statistics (class 3)

Before performing these exercises see the video “Revisions of statistics” (revstat)

9. The purpose of this exercise is to analyze the distribution of packet sizes observed at a network interface of your computer, using Wireshark and R. Using Wireshark, capture traffic in 3 different situations:
 - a) While watching a video on YouTube
 - b) While downloading a large file
 - c) While doing nothing

Then use R to analyze the data. Explain how the differences in packet sizes correlate with the application types. **Hints:** You may download a large file from this page: <https://www.thinkbroadband.com/download>. Use the Export Packet Dissections option of Wireshark to save the capture in CSV format. Then, use the `read.csv()` function to import the data into R. You may see some packet size values higher than the maximum packet size supported by your access network; this is due to so-called TCP Segmentation Offload. Remove these packets from the data. For example, to remove from vector `a` the elements that have a value higher than 100 use the instruction `a = a[!which(a > 100)]`; make sure you understand this instruction. Analyze the packet sizes using the `hist()` function. To zoom in a specific range of the histogram use the `xlim` parameter of the `hist()` function.

10. Perform a ping to a distant country (e.g., New Zealand) to measure 10 Round Trip Times (RTTs). Determine two 95% confidence intervals for the average RTT, one based on the Student's t distribution and the other based on the Normal distribution. Which interval do you prefer? **Hint:** The critical points of the Normal and the Student's t distributions can be obtained through the `qnorm()` and `qt()` functions of R, respectively.
11. Consider the following four distributions: standard Normal (with $\mu = 0$ and $\sigma^2 = 1$), Exponential with $\lambda = 1$, Lognormal with $\mu = 0$ and $\sigma^2 = 1$, and Lognormal with $\mu = 0$ and $\sigma^2 = 2$. For each distribution, and for sample sizes of $n = 5, 10, 50$ and 5000 , estimate the coverage of the 95% confidence interval for the mean based on 1000 experiments. Note that the mean of the lognormal is $e^{(\mu + \sigma^2/2)}$. What can you conclude? Are all confidence intervals trustful? Note that, in practice, the population mean is unknown; this is just an exercise to assess the behavior of confidence intervals. **Hint:** Use as an example the script `coveragenormal.R`, which obtains the coverage for the case of the Normal distribution.

F. Output data analysis (class 4)

Before performing these exercises see the video "Output data analysis" (outdata)

12. Consider the simulator for estimating the average delay in queue of an M/M/1 system whose R code is presented in Annex A. Run the simulation 10 times, with the stopping condition `NumQueueCompleted < 1000` and $\lambda/\mu = 0.8$. Compare the values obtained in each simulation run. Is one simulation run enough to draw conclusions?
13. The goal of this exercise is to evaluate the impact of the initial conditions in the performance estimates. Execute 25 runs of the M/M/1 simulation with $\lambda = 3$ and $\mu = 4$, using two different run lengths (20 and 2000), and two different initial conditions (0 and 10 clients initially in the system). For each of these four cases, compute the mean and the 95% confidence interval of the average delay in queue. The run length is the number of computed queuing delays in each run. Use the function `fmm1_initcond()` available in the `initcond.R` script. What can you conclude regarding the effect of the initial conditions in the estimate of the average delay in queue?

14. Using the replication/deletion procedure applied to the estimation of the average queuing delay of an M/M/1 system, analyze the number of required replicas as a function of (i) relative error, (ii) number of delays used to compute the actual estimate, and (iii) the ratio λ/μ . Use only values such that $\lambda/\mu < 1$. Use the function `frepdel()` available in the `repdel.R` script. **Hint:** *To better understand the results relative to $\rho = \lambda/\mu$, plot the queuing delay variance as a function of ρ (see the formula in Annex B).*

References

- DDRS course notes, chapter 1 and 3, Rui Valadas
- “Simulation, Modeling and Analysis”, Averill M. Law, McGraw-Hill, 4th edition, 2007

Duration

See the DDRS roadmap.

Work guidelines

The code that you develop in R must be well structured, simple, and easy to read. You must insert comments in your code to improve readability.

Visualization

All exercises must be demonstrated to the teacher during the classes.

Report

See the DDRS roadmap for instructions on how to write the report.

Plagiarism

Any copy detected between reports leads to cancelling all the involved reports, and a very angry teacher.

Extra (optional) exercises

15. Urn 1 contains four blue balls and two yellow ones, while in Urn 2 the mix is three blue and five yellow. A ball is drawn at random from Urn 1, and then put into Urn 2. A ball is then drawn at random from Urn 2. Develop an R script to estimate the probability that the second ball drawn is blue and compare the result with the exact value. **Hint:** Use the law of total probability to obtain the exact value.
16. Analyze the quality of the relative frequency as an estimator of the success probability of the Bernoulli random variable. Perform a large number of sets of experiments (say 500) where, in each set, N Bernoulli numbers are generated, and the relative frequency of successes is determined. Then, calculate the mean and the variance of the relative frequency of successes and compare with the theoretical values. Repeat the above procedure for different values of N , say 100, 500, 1000, and 10000. How does the quality of the estimate vary with N ?
17. The script `plotbern.R` generates a sequence of N Bernoulli numbers with success probability p , and plots the relative frequency of successes as a function of N . Repeat the plot several times, and for different values of p . What can you conclude regarding the possibility of having the relative frequency as an estimator of the success probability?
18. Consider the following sample from an unknown population having 12 points: 1.58, -1.04, 2.11, 0.41, 3.40, -0.34, -0.12, 3.14, 3.12, 1.54, 1.53, 2.76. Determine two 95% confidence intervals for μ , one based on the Student's t distribution and the other based on the Normal distribution. If these points are results of simulation runs, which interval do you prefer? **Hint:** The critical points of the Normal and the Student's t distributions can be obtained through the `qnorm()` and `qt()` functions of R, respectively.

Annex A. R code of the simulator that estimates the average delay in queue of a M/M/1 queuing system

```
ArrivalRate=1
ServiceRate=2
Time=0
NumQueueCompleted=0
ServerStatus=0
NumInQueue=0
AcumDelay=0
QueueArrivalTime=c()
EventList=c(rexp(1,ArrivalRate),Inf)
while (NumQueueCompleted<1000) {
  NextEventType=which.min(EventList)
  Time=EventList[NextEventType]
  if (NextEventType==1) {
    EventList[1]=Time+rexp(1,ArrivalRate)
    if (ServerStatus==1) {
      QueueArrivalTime=c(QueueArrivalTime,Time)
      NumInQueue=NumInQueue+1
    } else {
      NumQueueCompleted=NumQueueCompleted+1
      ServerStatus=1
      EventList[2]=Time+rexp(1,ServiceRate)
    }
  } else {
    if (NumInQueue==0) {
      ServerStatus=0
      EventList[2]=Inf
    } else {
      AcumDelay=AcumDelay+Time-QueueArrivalTime[1]
      QueueArrivalTime=QueueArrivalTime[-1]
      NumInQueue=NumInQueue-1
      NumQueueCompleted=NumQueueCompleted+1
      EventList[2]=Time+rexp(1,ServiceRate)
    }
  }
}
AvgDelay=AcumDelay/NumQueueCompleted
```

Annex B. Performance metrics of the M/M/1 queuing system

Average number of clients in system: $L = \frac{\lambda}{\mu - \lambda}$

Average delay in system: $W = \frac{1}{\mu - \lambda}$

Average delay in queue: $W_Q = \frac{\lambda}{\mu(\mu - \lambda)}$

Average number of clients in queue: $L_Q = \frac{\lambda^2}{\mu(\mu - \lambda)}$

Average number of clients in server: $L_S = \frac{\lambda}{\mu}$

Variance of delay in queue: $\sigma_{W_Q}^2 = \frac{\rho(2-\rho)}{\mu^2(1-\rho)^2}$

Annex C. Performance metrics of the M/M/m queuing system

Utilization factor: $\rho = \lambda/m\mu$

Probability that an arrival finds all servers busy: $P_Q = \frac{(m\rho)^m}{m!(1-\rho) \left[\left(\sum_{i=0}^{m-1} \frac{(m\rho)^i}{i!} \right) + \frac{(m\rho)^m}{m!(1-\rho)} \right]}$

Average number of clients in queue: $L_Q = \sum_{n=0}^{\infty} n\pi_n \frac{m^m \rho^{m+n}}{m!} = P_Q \frac{\rho}{1-\rho}$

Average delay in queue: $W_Q = \frac{L_Q}{\lambda} = P_Q \frac{\rho}{\lambda(1-\rho)}$

Average delay in system: $W = \frac{1}{\mu} + W_Q = \frac{1}{\mu} + \frac{P_Q}{m\mu - \lambda}$

Average number of clients in system: $L = \lambda W = m\rho + \frac{\rho P_Q}{1-\rho}$